

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

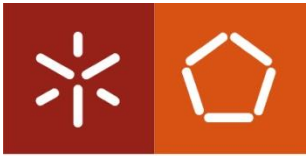
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

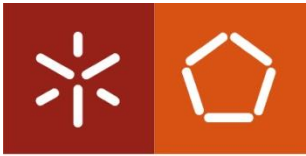
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães



Universidade do Minho

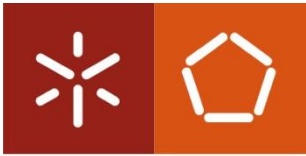
Sumário

- Algoritmos de ordenação
 - Insertion sort
 - Selection sort
 - Bubble Sort
 - Merge Sort
 - Quick Sort
- Análise da Complexidade



Ordenação de Vetores

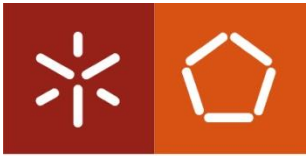
- Problema (*ordenação de vector*)
 - Dado um vector (v) com N elementos, reorganizar esses elementos por ordem crescente (ou melhor, por ordem não decrescente, porque podem existir valores repetidos)
- Ideias base:
 - Existem diversos algoritmos de ordenação (“sorting”) com complexidade $O(N^2)$ - por exemplo Ordenação por Inserção ou BubbleSort que são muito simples
 - Existem algoritmos de ordenação mais difíceis de codificar que têm complexidade $O(N \log N)$



Universidade do Minho

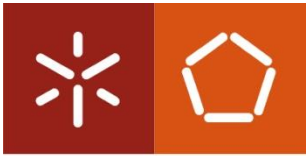
Algoritmos de Ordenação de Vetores

- Algoritmos:
 - Algoritmos fáceis de implementar, mas pouco eficientes:
 - Ordenação por Inserção
 - Ordenação por Seleção
 - BubbleSort
 - Algoritmos menos fáceis de implementar mas muito eficientes:
 - MergeSort
 - Ordenação por Partição (QuickSort)



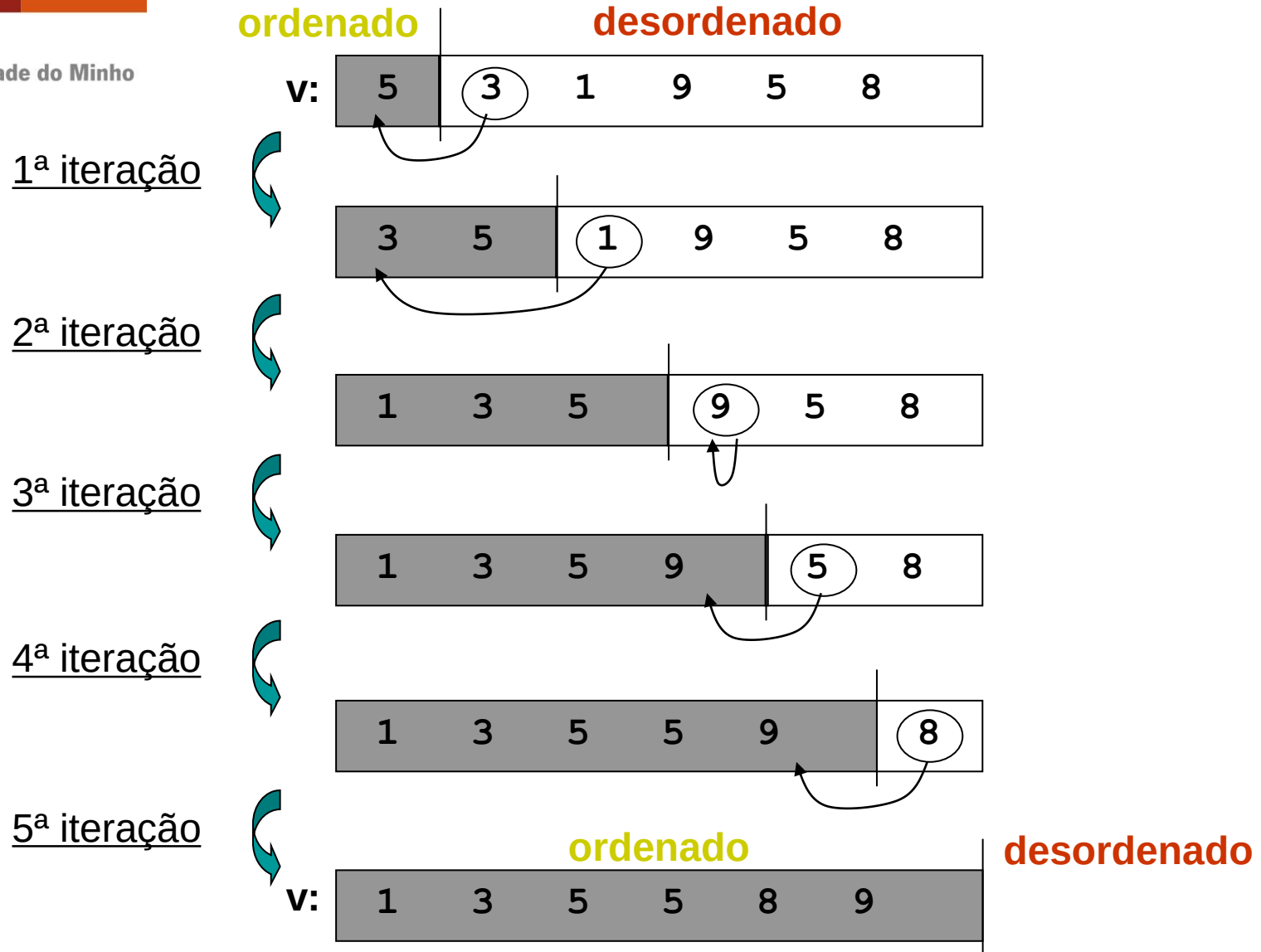
Ordenação por Inserção

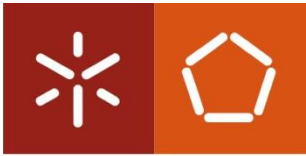
- **N-1** passos
- Em cada passo p coloca um elemento na ordem, sabendo que elementos dos índices inferiores (entre **0** e **p-1**) já estão ordenados
- Algoritmo (*ordenação por inserção*):
 - Considera-se o vetor dividido em dois sub-vetores (esquerdo e direito), com o da esquerda ordenado e o da direita desordenado
 - Começa-se com um elemento apenas no sub-vetor da esquerda
 - Move-se um elemento de cada vez do sub-vetor da direita para o sub-vetor da esquerda, inserindo-o na posição correta por forma a manter o sub-vetor da esquerda ordenado
 - Termina-se quando o sub-vetor da direita fica vazio



Universidade do Minho

Exemplo de Ordenação por Inserção



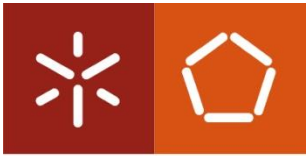


Inserção de valor em vetor ordenado

Universidade do Minho

- Sub-problema: inserir um valor num vetor ordenado (mantendo-o ordenado)
- Solução em C (função `InsertSorted`) :

```
/* Insere um valor x num array vec com n elementos ordenados.  
Após a inserção, o array fica com n+1 elementos ordenados.  
Retorna a posição em que inseriu o valor (entre 0 e n). */  
int InsertSorted(int vec[], int n, int x)  
{  
    int j;  
    /* procura posição (j) de inserção da direita (posição n)  
       para a esquerda (posição 0), e ao mesmo tempo chega  
       elementos à direita (podia usar o próprio n em vez de j) */  
    for (j = n; j > 0 && x < vec[j-1]; j--)  
        vec[j] = vec[j-1];  
    vec[j] = x; // insere agora  
    return j; // retorna a posição em que inseriu  
}
```

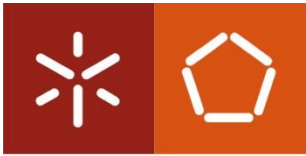


Implementação da Ordenação por Inserção em C

Universidade do Minho

- A função em C para ordenar um array pelo método de inserção é agora trivial

```
/* Ordena array vec de n elementos:  $vec[0] \leq \dots \leq vec[n-1]$  */  
void InsertionSort(int vec[], int n)  
{  
    /* i - tamanho do sub-array esquerdo ordenado */  
    int i;  
    for (i = 1; i < n; i++)  
        InsertSorted(vec, i , vec[i]);  
}
```

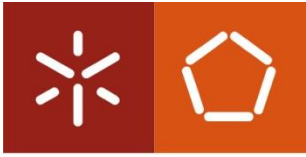



Implementação da Ordenação por Inserção em C

Universidade do Minho

- Juntando tudo (para ser mais eficiente) ...

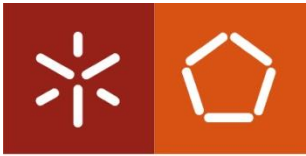
```
void InsertionSort(int vec[], int n)
{
    int i, j, x;
    for (i = 1; i < n; i++) {
        x = vec[i];
        for (j = i; j > 0 && x < vec[j-1]; j--)
            vec[j] = vec[j-1];
        vec[j] = x;
    }
}
```



Universidade do Minho

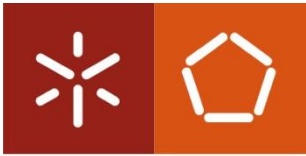
Análise da Ordenação por Inserção

- 2 ciclos encaixados, cada um pode ter N iterações :
 - $O(N^2)$
- Caso mais desfavorável: vetor em ordem inversa
 - $O(N^2)$
- Caso mais favorável: vetor já ordenado
 - $O(N)$
- **Conclusão:**
 - Só pode ser utilizado para vetores pequenos...



Ordenação por Seleção

- Provavelmente é o algoritmo mais intuitivo:
 - Encontrar o mínimo do vetor
 - Trocar com o primeiro elemento
 - Continuar para o resto do vetor (excluindo o primeiro)
- 2 ciclos encaixados, cada um pode ter N iterações :
 - **Complexidade $O(N^2)$**
- Variantes:
 - “Stable Sort” – Insere mínimo na primeira posição (em vez de realizar a troca)
 - “Shaker Sort” – Procura máximo e mínimo em cada iteração (Seleccção bidireccional)



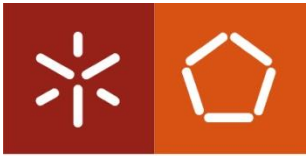
Ordenação por Seleção

Universidade do Minho

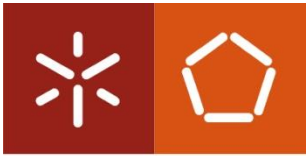
Algoritmo em C:

/ Ordena array **vec** de **n** elementos inteiros, ficando $vec[0] \leq \dots \leq vec[n-1]$ usando ordenação por seleção*/*

```
void selectionSort(int vec[], int tam)
{
    int i, j, min, aux;
    for (i=0; i<tam-1; i++) {
        min = i;
        for (j=i+1; j<tam; j++) {
            if (vec[j] < vec[min])
                min = j;
        }
        aux = vec[i];
        vec[i] = vec[min];
        vec[min] = aux;
    }
}
```



- Algoritmo de Ordenação “BubbleSort” (“Exchange Sort”):
 - Compara elementos adjacentes. Se o segundo for menor do que o primeiro, troca-os
 - Fazer isto desde o primeiro até ao último par
 - Repetir para todos os elementos exceto o último (que já está correto)
 - Repetir, usando menos um par em cada iteração até não haver mais pares (ou não haver trocas)
- Diversas variantes
- 2 ciclos encaixados, cada um pode ter N iterações:
 - **Complexidade $O(N^2)$**

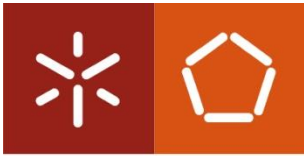


Universidade do Minho

Algoritmo em C:

/ Ordena array **vec** de **n** elementos inteiros, ficando $vec[0] \leq \dots \leq vec[n-1]$ usando BubbleSort */*

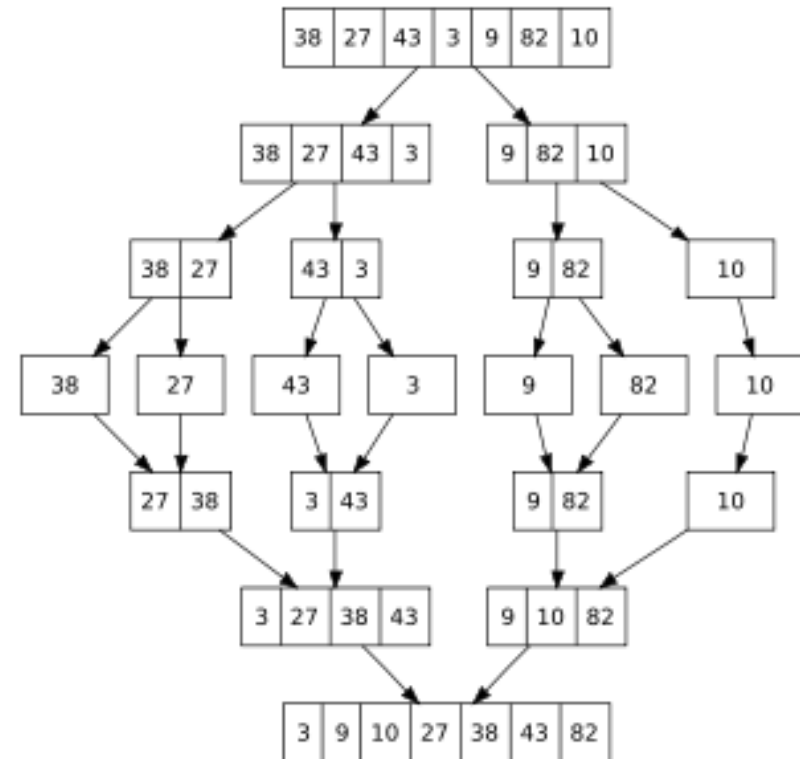
```
void bubble(int vec[], int n)
{
    int troca, i, j, aux;
    for (i = n-1; i > 0; i--) {
        /* o maior valor entre vec[0] e vec[i] vai para a posição vec[i]*/
        troca = 0;
        for (j = 0; j < i; j++) {
            if (vec[j] > vec[j+1]) {
                aux = vec[j]; vec[j] = vec[j+1]; vec[j+1] = aux;
                troca = 1;
            }
        }
        if (!troca) return;
    }
}
```



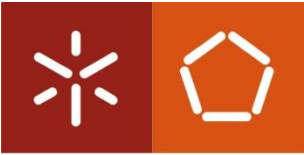
Universidade do Minho

MergeSort

- Abordagem recursiva
 - Divide-se o vetor ao meio
 - Ordena-se cada metade (usando MergeSort recursivamente)
 - Fundem-se as duas metades já ordenadas
- Divisão e Conquista:
 - Problema é dividido em dois de metade do tamanho
- Análise
 - Tempo execução: $O(N \log N)$
 - 2 chamadas recursivas de tamanho $N/2$
 - Operação de junção de vectores: $O(N)$
- Inconveniente
 - fusão de vectores requer espaço extra linear



Fonte: Wikipedia



MergeSort

Universidade do Minho

*/*Algoritmo em C */*

```
void mergeSort(int vec[], int n)
```

```
{
```

```
    int tmpVec[n];
```

```
    mergeSort2(vec, tmpVec, 0, n-1);
```

```
}
```

/ Método que realiza chamadas recursivas:*

vec é um vetor de elementos do tipo int.

tmpVec é um vetor para colocar o resultado da fusão (merge)

*left (right) é o elemento mais à esquerda(direita) do sub vetor */*

```
void mergeSort2(int vec[], int tmpVec[], int left, int right)
```

```
{
```

```
    int center;
```

```
    if (left < right) {
```

```
        center = (left + right)/2;    /*divide ao meio */
```

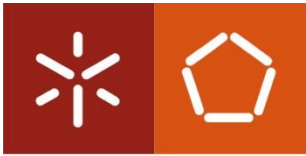
```
        mergeSort2(vec, tmpVec, left, center);    /* ordena 1ª metade */
```

```
        mergeSort2(vec, tmpVec, center+1, right); /*ordena 2ª metade*/
```

```
        merge(vec, tmpVec, left, center+1, right); /* Junta metades*/
```

```
    }
```

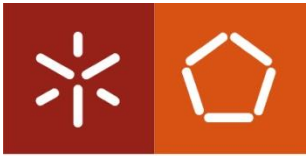
```
}
```

MergeSort

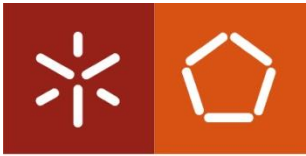
Universidade do Minho

```
/* Algoritmo de fusão dos dois vetores do MergeSort*/
void merge(int vec[], int tmpVec[],
           int leftPos, int rightPos, int rightEnd)
{
    int i;
    int leftEnd = rightPos - 1;
    int tmpPos = leftPos;
    int numElements = rightEnd - leftPos + 1;
    while (leftPos <= leftEnd && rightPos <= rightEnd )
        if (vec[leftPos] <= vec[rightPos] )
            tmpVec[tmpPos++] = vec[leftPos++];
        else
            tmpVec[tmpPos++] = vec[rightPos++];
    while (leftPos<=leftEnd) tmpVec[tmpPos++] = vec[leftPos++];
    while (rightPos<=rightEnd) tmpVec[tmpPos++] = vec[rightPos++];
    for ( i = 0; i < num Elements; i++, rightEnd-- )
        vec[rightEnd] = tmpVec[rightEnd];
}
```



Ordenação por Partição (Quick Sort)

- Algoritmo (ordenação por partição):
 1. Caso básico: Se o número (n) de elementos do vector (v) a ordenar for 0 ou 1, não é preciso fazer nada
 2. Passo de partição:
 - 2.1. Escolher um elemento arbitrário (x) do vector (chamado pivot)
 - 2.2. Partir o vector inicial em dois sub-vectores (esquerdo e direito), com valores $\leq x$ no sub-vector esquerdo e valores $\geq x$ no sub-vector direito (podendo existir um 3º sub-vector central com valores $=x$)
 3. Passo recursivo: Ordenar os sub-vectores esquerdo e direito, usando o mesmo método recursivamente
- Algoritmo recursivo baseado na técnica *divisão e conquista*



Refinamento do Passo de Partição

Universidade do Minho

(Supõe-se excluído pelo menos o caso em que $n = 0$)

2. Passo de partição:

2.1. Escolher para pivot (x) o elemento do meio do vector ($vec[n/2]$)

2.2. Inicializar $i = 0$ (índice da 1ª posição do vector)

Inicializar $j = n-1$ (índice da última posição do vector)

2.3. Enquanto $i \leq j$, fazer:

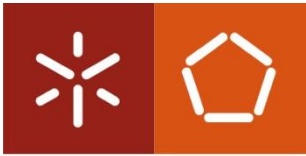
2.3.1. Enquanto $vec[i] < x$ (é sempre $i \leq n-1$!), incrementar i

2.3.2. Enquanto $vec[j] > x$ (é sempre $j \geq 0$!), decrementar j

2.3.3. Se $i \leq j$ então trocar $vec[i]$ com $vec[j]$, incrementar i e decrementar j

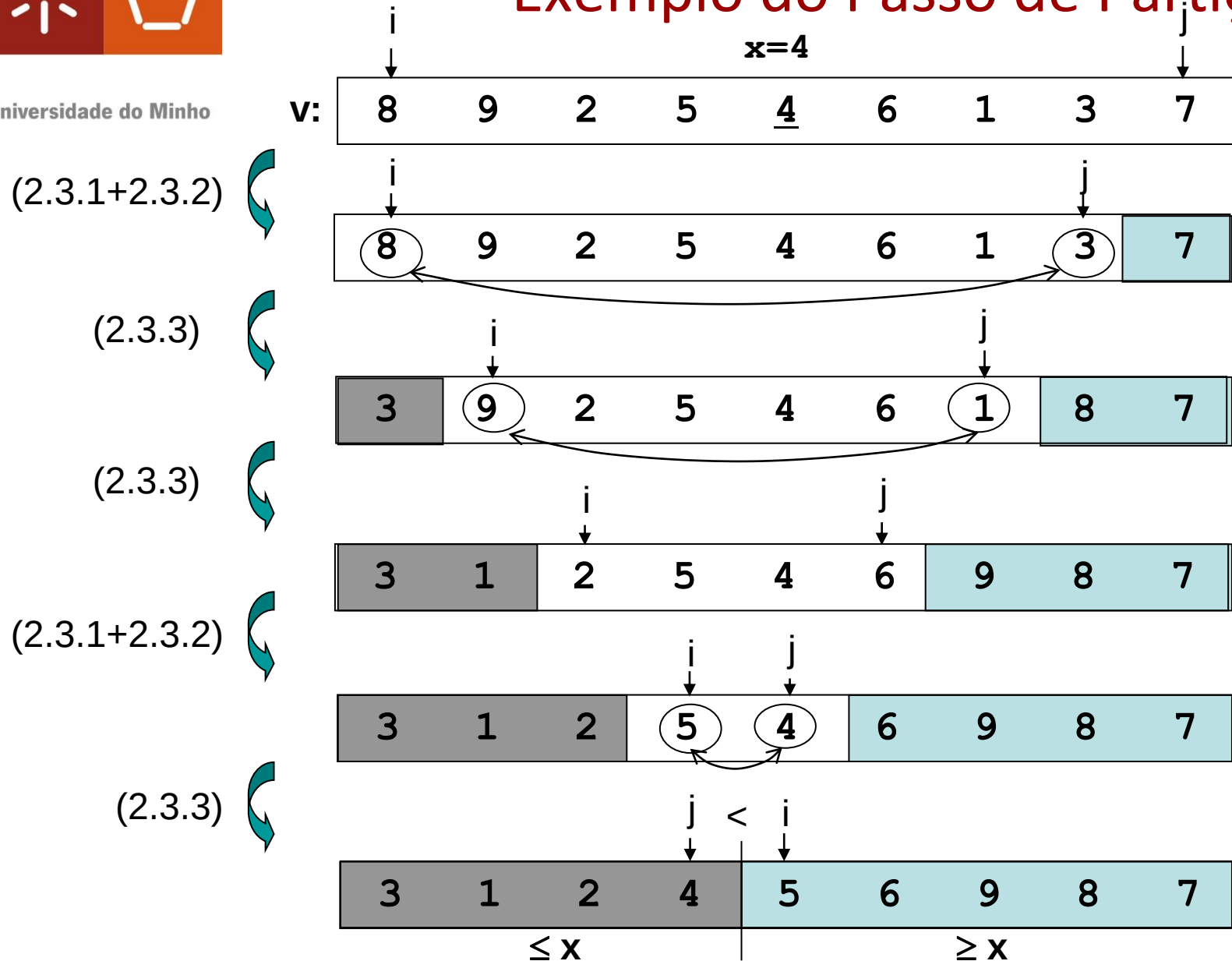
2.4. O sub-vector esquerdo (com valores $\leq x$) é $vec[0], \dots, vec[j]$ (vazio se $j < 0$)

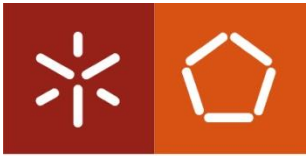
2.5. O sub-vector direito (com valores $\geq x$) é $vec[i], \dots, vec[n-1]$ (vazio se $i > n-1$)



Universidade do Minho

Exemplo do Passo de Partição

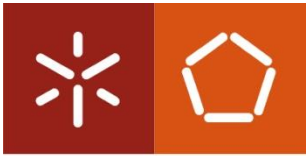




Implementação da Ordenação por Partição em C

Universidade do Minho

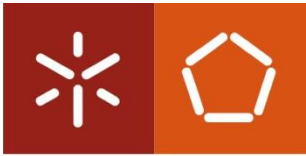
```
/* Ordena array(vec) entre 2 posições (a e b). Supõe que os elementos
do array são comparáveis com "<" e ">" e copiáveis com "=". */
void QuickSort(int vec[], int a, int b)
{
    if (a >= b) return; /* caso básico (tamanho <= 1) */
    int x = vec[(a+b)/2], tmp;
    int i = a, j = b;
    // passo de partição
    do {
        while (vec[i] < x) i++;
        while (vec[j] > x) j--;
        if (i > j) break;
        tmp=vec[i]; vec[i]=vec[j]; vec[j]=tmp;
        i++; j--; /*troca*/
    } while (i <= j);
    /* passo recursivo */
    QuickSort(vec, a, j);
    QuickSort(vec, i, b);
}
```



Análise da Ordenação por Partição

Universidade do Minho

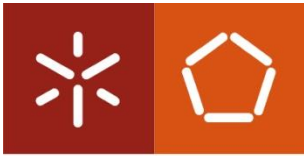
- Escolha pivot determina eficiência
 - *pior caso*: pivot é elemento mais pequeno
 $O(N^2)$
 - *melhor caso*: pivot é elemento médio
 $O(N \log N)$
 - *caso médio*: pivot corta vector arbitrariamente
 $O(N \log N)$
- Escolha do pivot
 - má escolha: extremos do vector ($O(N^2)$ se vector ordenado)
 - aleatório: envolve mais uma função pesada
 - recomendado: mediana de três elementos (extremos do vector e ponto médio)



Eficiência da Ordenação por Partição

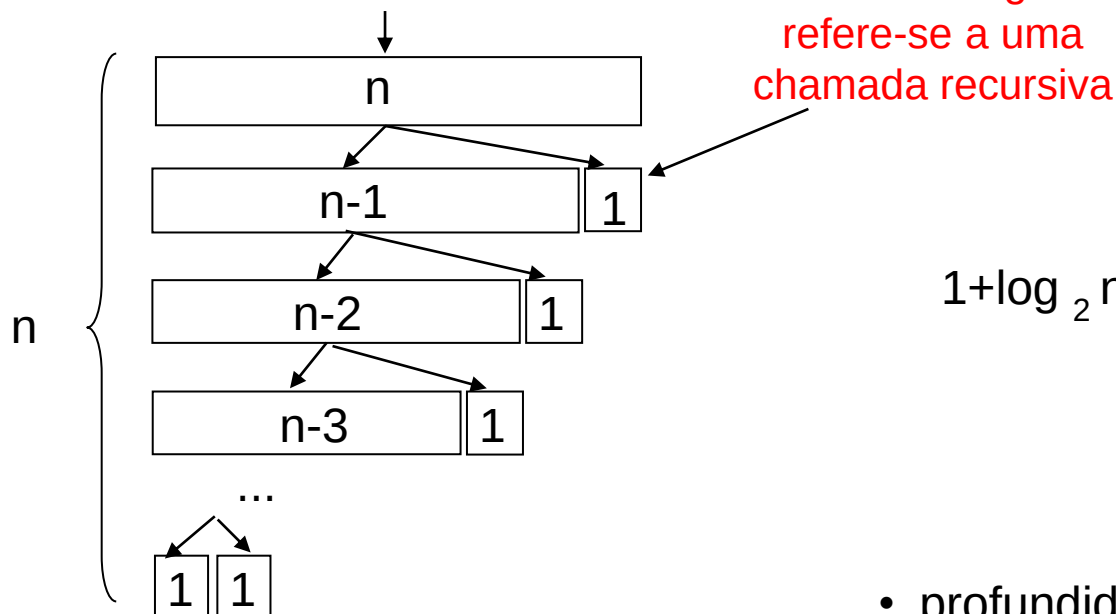
Universidade do Minho

- As operações realizadas mais vezes no passo de partição são as comparações efetuadas nos passos 2.3.1 e 2.3.2.
- No conjunto dos dois passos, o número de comparações efetuadas é:
 - no mínimo n (porque todas as posições do array são analisadas)
 - no máximo $n+2$ (correspondente à situação em que $i=j+1$ no fim do passo 2.3.2)
- Por conseguinte, o tempo de execução do passo de partição é $O(n)$
- Para obter o tempo de execução do algoritmo completo, é necessário somar os tempos de execução do passo de partição, para o array inicial e para todos os sub-arrays aos quais o algoritmo é aplicado recursivamente



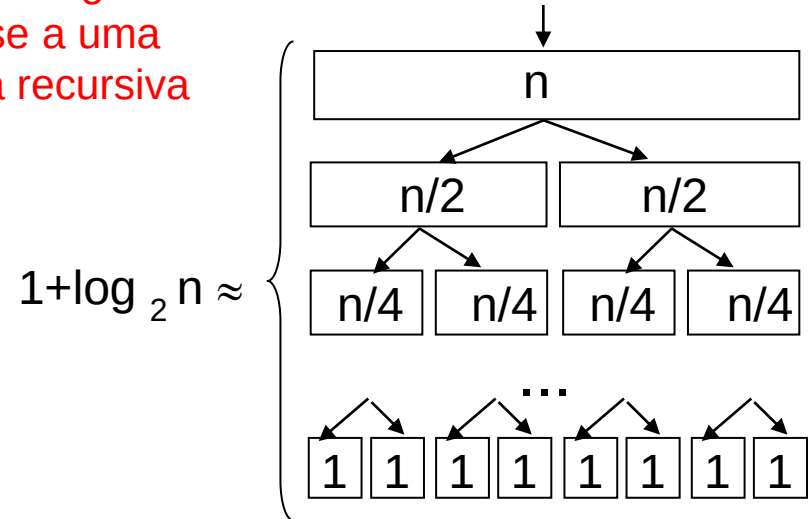
Eficiência da Ordenação por Partição (cont.)

Pior caso:

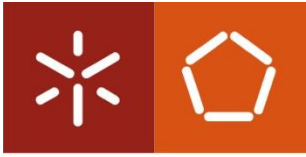


- profundidade de recursão: n
- tempo de execução total (somando totais de linhas):
 $T(n) = O[n + n + (n-1) + \dots + 2]$
 $= O[n + (n-1)(n+2)/2] = O(n^2)$

Melhor caso:



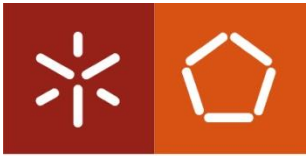
- profundidade de recursão: $\approx 1 + \log_2 n$ (sem contar com a possibilidade de um elemento ser excluído dos sub-arrays esquerdo e direito)
- tempo de execução total (uma vez que a soma de cada linha é n):
 $T(n) = O[(1 + \log_2 n) n] = O(n \log n)$



Universidade do Minho

Eficiência da Ordenação por Partição (cont.)

- Prova-se que no caso médio (na hipótese de os valores estarem aleatoriamente distribuídos pelo array), o tempo de execução é da mesma ordem que no melhor caso, isto é:
$$T(n) = O(n \log n)$$
- O critério seguido para a escolha do *pivot* destina-se a tratar eficientemente os casos em que o array está inicialmente ordenado



Algoritmos de Ordenação

Universidade do Minho

- Cada ponto corresponde à ordenação de 100 vetores de inteiros gerados aleatoriamente
(Fonte: Sahni, "Data Structures, Algorithms and Applications in C++")
- Método de ordenação por partição
(*quickSort*) é na prática o mais eficiente, exceto para arrays pequenos (até cerca 20 elementos),
em que o método de ordenação por inserção (*insertionSort*) é melhor!

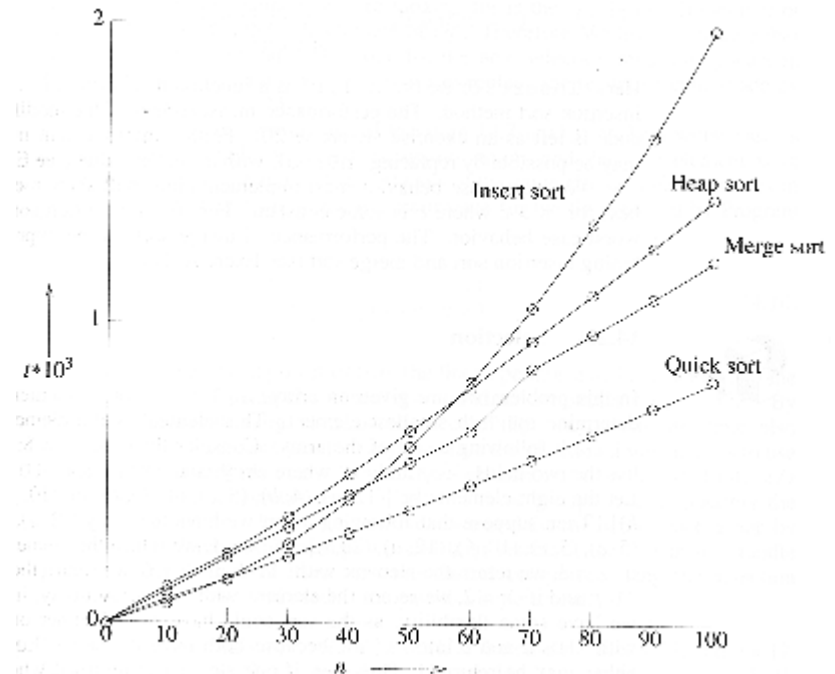


Figure 14.12 Plot of average times